

Práctica: Persistencia del dominio con EF Core y migraciones

Paso a paso — Configurar DbContext, Fluent API y aplicar la primera migración de base de datos

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 4 — Persistencia con Entity Framework Core
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Básico (guía principal de la sesión)
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Instalar y configurar Entity Framework Core 8 con proveedor SQLite en el proyecto Infrastructure.
- Crear la clase SolicitudesDbContext exponiendo los DbSet correspondientes a las entidades de dominio.
- Mapear entidades con Fluent API (EntityTypeConfiguration) manteniendo la pureza de la capa Domain.
- Generar la migración inicial mediante la CLI de .NET y aplicarla sobre la base de datos.
- Verificar la creación física de las tablas relacionales y sus restricciones de clave primaria y foránea.

2. Desarrollo paso a paso

Paso 1 — Instalar paquetes de EF Core

Instalamos los paquetes oficiales de EF Core en Infrastructure y las herramientas de diseño en la API.

terminal

```
dotnet add src/SolicitudesApp.Infrastructure package Microsoft.EntityFrameworkCore.Sqlite
dotnet add src/SolicitudesApp.Infrastructure package Microsoft.EntityFrameworkCore.Design
dotnet add src/SolicitudesApp.Api package Microsoft.EntityFrameworkCore.Design
```

Paso 2 — Crear SolicitudesDbContext

En Infrastructure/Persistencia/SolicitudesDbContext.cs, declaramos la clase que administra la conexión y los conjuntos de entidades.

SolicitudesDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infrastructure.Persistencia;

public class SolicitudesDbContext : DbContext
{
    public SolicitudesDbContext(DbContextOptions<SolicitudesDbContext> options) : base(options) {

        public DbSet<Solicitud> Solicitudes => Set<Solicitud>();
        public DbSet<SolicitudDetalle> SolicitudDetalles => Set<SolicitudDetalle>();
        public DbSet<Usuario> Usuarios => Set<Usuario>();
        public DbSet<Rol> Roles => Set<Rol>();
        public DbSet<Aprobacion> Aprobaciones => Set<Aprobacion>();
        public DbSet<MovimientoTesoreria> MovimientosTesoreria => Set<MovimientoTesoreria>();
        public DbSet<CentroCosto> CentrosCosto => Set<CentroCosto>();

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.ApplyConfigurationsFromAssembly(typeof(SolicitudesDbContext).Assembly);
        }
    }
}
```

ApplyConfigurationsFromAssembly: busca y registra automáticamente todas las clases que implementan `EntityTypeConfiguration` en el proyecto, evitando tener que registrarlas una a una a mano.

Paso 3 — Configurar la entidad Solicitud con Fluent API

Creamos `SolicitudConfiguration.cs` para definir longitudes, tipos de columna y precisión monetaria.

SolicitudConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infraestructura.Persistencia.Configuraciones;

public class SolicitudConfiguration : IEntityTypeConfiguration<Solicitud>
{
    public void Configure(EntityTypeBuilder<Solicitud> builder)
    {
        builder.ToTable("Solicitudes");
        builder.HasKey(s => s.Id);

        builder.Property(s => s.Motivo).IsRequired().HasMaxLength(250);
        builder.Property(s => s.MontoTotal).HasPrecision(18, 2);
        builder.Property(s => s.Estado).HasConversion<int>();

        // Relación 1 a N con SolicitudDetalle (Cascade delete)
        builder.HasMany(s => s.Detalles)
            .WithOne(d => d.Solicitud)
            .HasForeignKey(d => d.SolicitudId)
            .OnDelete(DeleteBehavior.Cascade);

        // Relación con el Solicitante (Restrict delete para proteger integridad)
        builder.HasOne(s => s.Solicitante)
            .WithMany()
            .HasForeignKey(s => s.SolicitanteId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}
```

Paso 4 — Configurar la cadena de conexión y registrar el servicio

Añadimos la cadena de conexión en appsettings.json y registramos el DbContext en Program.cs.

Program.cs

```
// appsettings.json:
// "ConnectionStrings": { "DefaultConnection": "Data Source=solicitudes.db" }

var connectionString = builder.Configuration.GetConnectionString("DefaultConnection")
    ?? "Data Source=solicitudes.db";

builder.Services.AddDbContext<SolicitudesDbContext>(options =>
{
    options.UseSqlite(connectionString);
});
```

Paso 5 — Crear y aplicar la migración inicial

Generamos el script de migración en C# y aplicamos los cambios sobre el archivo de base de datos.

terminal

```
dotnet ef migrations add MigracionInicial --project src/SolicitudesApp.Infraestructura --startup-
project src/SolicitudesApp.Api
dotnet ef database update --project src/SolicitudesApp.Infraestructura --startup-project
src/SolicitudesApp.Api
```

3. Retos adicionales

1. Configura un índice único en la entidad Usuario para que no se puedan registrar dos usuarios con el mismo correo.
2. Implementa la configuración Fluent API para MovimientoTesoreria vinculando la relación 1 a 1 con Solicitud.
3. Inspecciona el script SQL que EF Core ejecuta con el comando `dotnet ef migrations script`.
4. Prueba deshacer la migración aplicando `dotnet ef database update 0` y vuelve a aplicarla.

4. Entregables

- La solución compila sin advertencias y el archivo de base de datos solicitudes.db se genera con las 7 tablas del dominio.
- Archivo de migración C# que describe las operaciones `Up()` y `Down()` del esquema relacional.
- Captura de pantalla o registro en terminal del comando `dotnet ef database update` completado en verde.
- Comentario en código explicando por qué se eligió `DeleteBehavior.Restrict` en la relación Usuario-Solicitud.

● Técnico de Desarrollo Full Stack con C# y Angular · Sesión 4 · Docente: Edwin Barrientos · New Horizons